



**UNIVERSITÀ
DI TRENTO**

Document: Analisi e Progettazione
Revision: 1.3

Dipartimento di Ingegneria e Scienze dell'informazione

Progetto:

UrbanLock

Gruppo:

G26

Titolo del documento:

Analisi e Progettazione

Doc. Name	D3-UrbanLockBusinessPlan	Doc. Number	D2 V 1.3
Description	Documento di design		

INDICE

Scopo del documento	2
1. Analisi dei Componenti	2
1.1 Definizione dei Componenti	2
1.2 Diagramma delle Componenti	10
2. Diagramma delle Classi	11
2.1 Diagramma delle classi complessivo	17
3. Dal class diagram alle API	18

Scopo del documento

Il presente documento riporta il design del progetto UrbanLock con l'utilizzo di un diagramma dei componenti e di un diagramma delle classi.

1. Analisi dei Componenti

1.1 Definizione dei Componenti

CMP1: Gestione Autenticazione e Ruoli

Descrizione:

Il componente gestisce l'accesso dei cittadini tramite SPID/CIE e l'accesso degli operatori tramite credenziali della console, generando una sessione autenticata con ruolo/permessi e tracciando i log di autenticazione.

Interfacce richieste – Richiesta login cittadino SPID/CIE (da App):

L'app invia la richiesta di login e la modalità scelta (SPID o CIE).

Il componente avvia il flusso e prepara la gestione di errori/timeout.

Interfacce richieste – Callback esito SPID/CIE (da SPID/CIE):

Il provider SPID/CIE restituisce al sistema l'esito dell'autenticazione.

Il componente usa l'esito per completare o negare l'accesso.

Interfacce richieste – Richiesta login operatore (credenziali console) (da Console):

La console invia credenziali operatore per accedere alle funzioni amministrative.

Il componente valida la richiesta e avvia la verifica verso il DB.

Interfacce richieste – Verifica credenziali operatore (a Database):

Interroga il DB per verificare credenziali e ruolo associato all'operatore.

Se fallisce, nega accesso; se riesce, crea sessione operatore.

Interfacce richieste – Lettura/creazione profilo utente (a Database):

Recupera il profilo associato all'identità autenticata o lo crea se assente.

Serve per mantenere dati e preferenze nel tempo.

Interfacce richieste – Scrittura log autenticazioni (a Database):

Scriva eventi di login/logout con data/ora ed esito per tracciamento.

I log seguono vincoli di privacy e conservazione stabiliti dalle policy.

Interfacce fornite – Esito login cittadino (a App):

Ritorna all'app successo/errore del login e lo stato della sessione.

Consente all'app di abilitare o bloccare l'accesso alle sezioni protette.

Interfacce fornite – Esito login operatore (a Console):

Ritorna alla console l'esito del login e abilita/disabilita le aree amministrative.

In caso di errore, evita la divulgazione di dettagli sensibili.

Interfacce fornite – Sessione autenticata (token + ruolo/permessi):

Fornisce un token/sessione da allegare alle chiamate verso componenti protetti.

Il token identifica utente/operatore e i permessi associati.

Interfacce fornite – Autorizzazione operatore (ok/ko + permessi):

Fornisce ai componenti admin un responso di autorizzazione per singole operazioni.

Uniforma l'applicazione dei permessi su tutte le funzioni di console.

CMP2: Mappa Lockers

Descrizione:

Il componente espone la ricerca e visualizzazione delle postazioni su mappa, con filtri e schede dettaglio, mostrando stato (online/offline/manutenzione) e disponibilità aggiornate.

Interfacce richieste – Parametri mappa e filtri (da App):

L'app invia distanza, tipologia locker, categoria contenuti, orari e preferenze.

Il componente usa tali parametri per produrre un set di risultati coerente.

Interfacce richieste – dettagli lockers e celle (da Gestione Lockers):

Richiede stati e disponibilità aggiornate di postazioni e celle.

Serve per disegnare correttamente pin e indicatori di disponibilità.

Interfacce fornite – Pin/marcatori mappa postazioni (a App):

Fornisce all'app la lista pin con coordinate e stato sintetico.

Include indicatori di disponibilità e condizioni di manutenzione/offline.

Interfacce fornite – Scheda dettaglio postazione (a App):

Fornisce dettagli su una postazione selezionata e lo stato delle celle.

Supporta la scelta dell'utente su dove recarsi e cosa usare.

CMP3: Gestione Lockers

Descrizione:

Il componente gestisce anagrafica e stato di postazioni/locker/celle e governa i flussi di utilizzo (apertura/restituzione), aggiornando stati e log che alimentano mappa, console e storico/report.

Interfacce richieste – Sessione autenticata (da Autenticazione):

Riceve token/sessione per associare operazioni a identità e tracciare eventi.
Senza sessione valida, le operazioni protette vengono negate.

Interfacce richieste – Autorizzazione operatore (da Autenticazione):

Richiede permessi per azioni amministrative su postazioni, categorie e celle commerciali.
Se non autorizzato, blocca l'operazione richiesta dalla console.

Interfacce richieste – Richiesta dettagli lockers e celle (da Mappa Lockers):

Riceve richiesta di elenco/dettaglio postazioni per alimentare la mappa.
Prepara dati coerenti con stati e disponibilità correnti.

Interfacce richieste – Richiesta apertura vano (da App):

Riceve richiesta apertura con identificativi postazione/locker/cella.
Avvia sblocco e registra l'evento di utilizzo.

Interfacce richieste – Richiesta restituzione vano (da App):

Riceve richiesta restituzione e attiva validazioni previste (foto o sensore).
Aggiorna disponibilità e stato a fine procedura.

Interfacce richieste – Lettura peso vano (da Locker fisico/IoT):

Riceve esito/valore del sensore peso per verificare corretta restituzione.
Associa il dato all'evento e lo rende tracciabile.

Interfacce richieste – dati lockers / celle (a Database):

Legge/scrive anagrafiche e stati, incluse manutenzioni e date ripristino.
Alimenta mappa, console e disponibilità verso i cittadini.

Interfacce fornite – Dettagli lockers e celle (da Mappa Lockers):

Fornisce dettagli completi per la scheda postazione (celle e stati).
Supporta anche filtri per tipologia e contenuti.

Interfacce fornite – Esito apertura vano (a App):

Restituisce esito apertura con messaggi di errore o conferma.
Aggiorna coerentemente la disponibilità della cella.

Interfacce fornite – Esito restituzione vano (a App):

Restituisce esito restituzione e conferma aggiornamento stato.
Dipende dal rispetto dei vincoli di validazione (foto/peso) quando previsti.

Interfacce fornite - modifiche amministrative locker (a Console):

- **Crea/modifica postazione e locker:**
Espone funzioni per creare o aggiornare postazioni e locker in inventario.
Consente la gestione operativa da parte del comune.

- **Gestione per tipologia:**
Espone strumenti per gestire tipologie diverse con procedure dedicate.
Permette differenziare rifornimento e manutenzione in base al tipo.
- **Imposta stato postazione:**
Espone la modifica dello stato online/offline/manutenzione e data ripristino.
Aggiorna DB e riflette la modifica su mappa e disponibilità.
- **Crea/modifica categoria locker:**
Espone creazione/aggiornamento categorie locker e validazioni necessarie.
Rende la categoria disponibile per future configurazioni.
- **Assegna/gestisci celle commerciali:**
Espone assegnazione, rinnovo e revoca delle celle destinate ai negozi.
Abilita gestione contrattuale “nel sistema” e tracciamento.

Interfacce fornite – Evento postazione (a Gestione Notifiche):

Invia eventi quando cambiano stati postazione o vengono aggiunte nuove postazioni.
Le notifiche possono informare gli utenti di chiusure temporanee e novità.

Interfacce fornite – Dati utilizzo per storico/report:

Rende disponibili dati di utilizzo e statistiche per consultazione e analisi.
L'accesso avviene con permessi e tracciamento coerenti con sicurezza/privacy.

CMP4: Gestione Donazioni

Descrizione:

Il componente gestisce la creazione e consultazione delle donazioni lato app e la lavorazione lato console (cambio stato), persistendo i dati e generando eventi per notifiche.

Interfacce richieste – Sessione autenticata (da Autenticazione):

Richiede una sessione valida per associare la donazione a un utente identificato.
Evita invii anonimi e permette tracciamento.

Interfacce richieste – Autorizzazione operatore (da Autenticazione):

Richiede permessi per consentire all'operatore di cambiare stato e lavorare le pratiche.
Blocca modifiche non autorizzate dalla console.

Interfacce richieste – Richiesta nuova donazione (da App):

Riceve i dati del modulo (informazioni e allegati come foto/descrizione).
Avvia validazioni e prepara la persistenza a DB.

Interfacce richieste – Richiesta elenco donazioni (da App):

Riceve richiesta di consultazione e filtra le donazioni appartenenti all'utente.
Restituisce lista aggiornata e stati correnti.

Interfacce richieste – Richiesta stato donazione (da App):

Riceve richiesta di dettaglio su una singola donazione (tracking).
Serve a mostrare avanzamento e prossimi passi (es. ritiro).

Interfacce richieste – Gestione donazioni: cambio stato (da Console):

Riceve dall'operatore le transizioni di stato e le azioni collegate (note/allegati/contatto).
Aggiorna la pratica e mantiene lo storico coerente.

Interfacce fornite – Elenco/stato donazioni (a App e Console):

Fornisce lista donazioni dell'utente con stati aggiornati.
Consente monitoraggio continuo senza contattare l'operatore.

Interfacce fornite – Dettaglio donazione (a Console):

Fornisce dettagli completi (dati utente, descrizione, allegati, storico stati).
Abilita decisioni operative (valutazione, ritiro, conclusione).

Interfacce fornite – Evento donazione (a Gestione Notifiche):

Genera eventi su cambio stato o reminder collegati alla donazione.
Consente al sistema di avvisare l'utente sugli step successivi.

CMP5: Gestione Segnalazioni

Descrizione:

Il componente riceve segnalazioni dagli utenti (con descrizione e foto) e fornisce agli operatori strumenti di moderazione (priorità, assegnazione, cambio stato), con persistenza e notifiche correlate.

Interfacce richieste – Sessione autenticata (da Autenticazione):

Richiede sessione valida per associare la segnalazione a un utente identificato.
Aiuta tracciamento e prevenzione abusi.

Interfacce richieste – Autorizzazione operatore (da Autenticazione):

Richiede permessi per moderare, assegnare priorità e aggiornare stato.
Impedisce che utenti non autorizzati modifichino i ticket.

Interfacce richieste – Invio segnalazione (da App):

Riceve descrizione, foto e riferimento a postazione/locker della segnalazione.
Valida campi obbligatori e crea il ticket.

Interfacce richieste – Lettura segnalazioni (a Database):

Legge elenco e dettagli segnalazioni, supporta ordinamenti e filtri per data/priorità/stato.

Interfacce fornite – Lista segnalazioni:

Fornisce all'operatore l'elenco completo delle segnalazioni con metadati utili.
Supporta prioritarizzazione e programmazione interventi.

Interfacce fornite – Evento segnalazione (a Gestione Notifiche):

Genera eventi per conferma invio e aggiornamenti di stato al cittadino.
Supporta comunicazioni automatiche e tracciabili.

CMP6: Gestione Profilo e Storico

Descrizione:

Il componente gestisce profilo utente, preferiti e preferenze di notifica, e rende disponibile lo storico utilizzo con statistiche lato app.

Interfacce richieste – Sessione autenticata (da Autenticazione):

Richiede sessione valida per leggere e modificare dati personali e preferenze.
Impedisce accessi non autorizzati ai dati dell'utente.

Interfacce richieste – Lettura profilo utente (a Database):

Legge dal DB i dati profilo dell'utente (informazioni applicative e impostazioni).
Serve a mostrare la schermata profilo in app.

Interfacce richieste – dati utente (a Database):

Legge i dati dell'utente, le preferenze e le impostazioni notifiche (abilitazioni/disabilitazioni, tipologie).

Interfacce richieste – Lettura storico utilizzo (a Database):

Legge i log di utilizzo registrati dai lockers per costruire lo storico utente.
Permette di calcolare statistiche (tempi, frequenze, preferenze).

Interfacce fornite – Dati profilo utente (a App):

Fornisce all'app i dati del profilo e le impostazioni salvate.
Consente visualizzazione e aggiornamento guidato.

Interfacce fornite – Storico utilizzo (a App):

Fornisce all'app l'elenco delle attività registrate (utilizzi e restituzioni).
Serve per trasparenza verso l'utente e riepilogo personale.

Interfacce fornite – Statistiche utente (a App):

Fornisce metriche derivate (es. tempo totale, fasce orarie ricorrenti).
Le statistiche sono basate sui dati storici memorizzati.

Interfacce fornite – Preferenze notifiche (a Gestione Notifiche):

Rende disponibili al componente notifiche le preferenze dell'utente per filtrare cosa inviare.
Può essere ottenuto tramite DB o tramite chiamata dedicata.

CMP7: Gestione Notifiche

Descrizione:

Il componente riceve eventi dai domini principali (lockers, donazioni, segnalazioni), applica le preferenze utente e richiede l'invio tramite provider esterno, tracciando gli invii.

Interfacce richieste – Eventi da Gestione Lockers:

Riceve eventi su aperture/chiusure, chiusure temporanee e nuove postazioni.
Gli eventi sono trasformati in notifiche secondo regole e preferenze.

Interfacce richieste – Eventi da Gestione Donazioni:

Riceve eventi su creazione donazione, cambi stato e reminder.
Permette di informare l'utente su avanzamenti e appuntamenti.

Interfacce richieste – Eventi da Gestione Segnalazioni:

Riceve eventi su conferma invio e cambi stato delle segnalazioni.
Consente aggiornamenti automatici all'utente senza intervento manuale.

Interfacce richieste – Lettura preferenze notifiche (a Database):

Legge dal DB le preferenze dell'utente per stabilire quali notifiche inviare.
Evita invii non desiderati e riduce rumore informativo.

Interfacce richieste – Scrittura log invii notifiche (a Database):

Scrive esito invio (ok/ko) e metadati minimi per tracciamento.
Supporta audit e diagnosi di problemi di consegna.

Interfacce richieste – Servizio invio push/email (verso Provider esterno):

Richiede a un provider esterno l'effettivo invio (push o email).
Gestisce esiti e possibili retry/logiche di consegna.

Interfacce fornite – Invio notifiche (verso provider):

Fornisce al provider contenuto, destinatario e tipo di notifica da inviare.
Standardizza il formato degli invii per tutti gli eventi.

Interfacce fornite – Log notifiche (verso Database):

Fornisce al DB i dati necessari a registrare invii e fallimenti.
Rende consultabile lo storico invii a fini di controllo.

CMP8: Gestione Reportistica

Descrizione:

Il componente consente all'operatore di generare report filtrati e di esportarli (PDF/Excel) a partire da dataset aggregabili presenti a database.

Interfacce richieste – Sessione autenticata (da Autenticazione):

Richiede token di sessione per eseguire operazioni in contesto operatore.
Serve a tracciare chi ha generato o esportato un report.

Interfacce richieste – Autorizzazione operatore (da Autenticazione):

Richiede permessi specifici per accedere alla reportistica e agli export.
Se non autorizzato, impedisce generazione e download.

Interfacce richieste – Richiesta report e statistiche (da Console):

La console invia i parametri di generazione report.
Avvia la raccolta dati e la costruzione delle viste.

Interfacce richieste – Lettura dataset donazioni (a Database):

Legge dati sulle donazioni per report e conteggi (quando inclusi).
Supporta analisi di andamento e stati di lavorazione.

Interfacce richieste – Lettura dataset segnalazioni (a Database):

Legge dati sulle segnalazioni per statistiche operative (quando incluse).
Aiuta a misurare volumi e priorità nel tempo.

Interfacce fornite – Dati aggregati report (a Console):

Fornisce alla console i risultati aggregati per la visualizzazione.
Permette di esplorare i dati con gli stessi filtri applicati.

CMP9: Gestione Database

Descrizione:

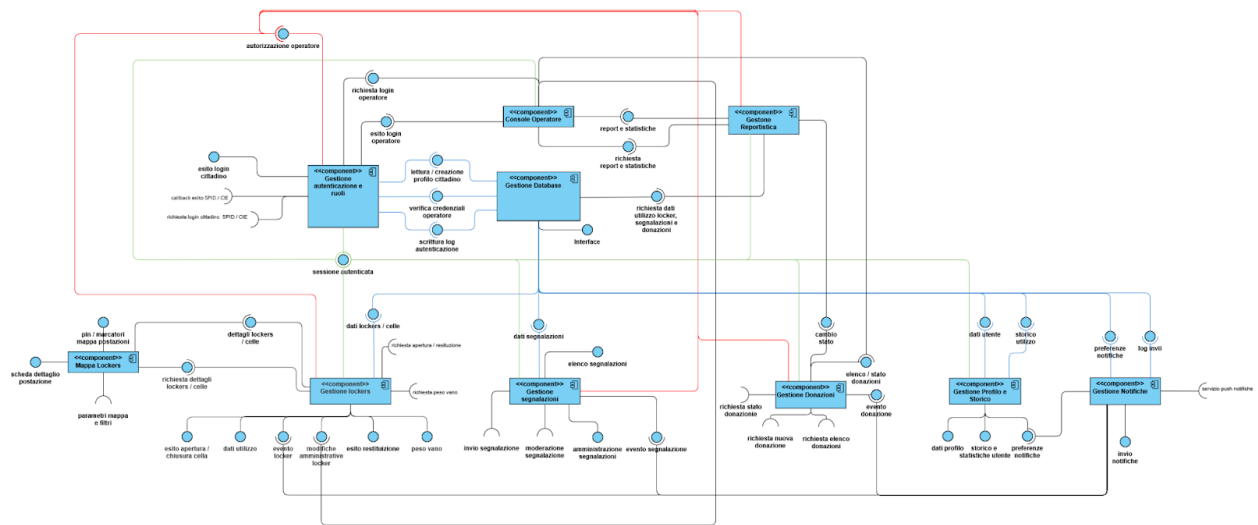
Il componente di persistenza fornisce operazioni di lettura/scrittura e query aggregate per tutti i domini applicativi e per la reportistica.

Interfacce fornite – Persistenza dati:

Fornisce operazioni di lettura/scrittura e query aggregate per tutti i dati del sistema (utenti/operatori/ruoli, log, postazioni/lockers/celle e relativi stati/manutenzioni/categorie, utilizzi e restituzioni, celle commerciali e storico accessi, donazioni, segnalazioni, preferiti, preferenze e log notifiche).

È l'interfaccia comune usata dagli altri componenti per mantenere informazioni coerenti e tracciabili nel tempo, a supporto di app, console, notifiche e reportistica.

1.2 Diagramma delle Componenti



Per vedere una foto migliore, [clicca qui](#).

2. Diagramma delle Classi

Nel presente capitolo vengono presentate le classi previste nell'ambito del progetto UrbanLock. Ogni componente presente nel diagramma dei componenti presentato nel capitolo precedente diventa una o più classi. Tutte le classi individuate sono caratterizzate da un nome, una lista di attributi che identificano i dati gestiti dalla classe e una lista di metodi che definiscono le operazioni previste all'interno della classe. Ogni classe può essere anche associata ad altre classi e, tramite questa associazione, è possibile fornire informazioni su come le classi si relazionano tra loro. Riportiamo di seguito le classi individuate, in questo processo si è proceduto anche nel massimizzare la coesione e minimizzare l'accoppiamento tra classi.

Utente

Abbiamo individuato l'attore "Utente", da cui la classe omonima, che è colui che può interagire con il sistema UrbanLock per accedere ai locker, gestire donazioni e visualizzare le proprie segnalazioni. L'utente è identificato da un codice fiscale univoco e possiede informazioni personali quali nome, cognome, indirizzo, residenza, numero di telefono ed email (informazioni prese nel momento in cui si esegue il login con SPID/CIE).

Utente

- CF: string
- nome: string
- cognome: string
- indirizzo: string
- telefono: int
- residenza: string
- email: string
- celle_usate
- + armadietti_on: Cella[celle_usate]

- + accedi()
- + visualizza_mappa()
- + apri_cella()
- + chiudi_cella()
- + storico_utilizzi(): return Noleggio[]
- + nuova_segnalazione()
- + nuova_donazione()
- +visualizza_locker(): return Cella[]
- + cambia_lingua()
- + cambia_tema()
- + logout()

Table 1: Dettaglio classe Utente

All'interno della classe utente sono state inserite le operazioni di accesso al sistema, visualizzazione della mappa dei locker disponibili, apertura delle celle noleggiate, consultazione dello storico degli utilizzi, creazione di nuove segnalazioni e donazioni, ricerca di locker specifici e gestione delle impostazioni personali come il cambio tema dell'applicazione.

Operatore

L'operatore è il personale addetto alla gestione operativa del sistemaUrbanLock che sfrutta il software per visualizzare lo stato dei locker, gestire le donazioni, gestire le segnalazioni e amministrare i locker stessi. L'operatore possiede credenziali di accesso (username e password) e può effettuare operazioni privilegiate non disponibili agli utenti comuni.

Operatore
- username: string - password: string
+ login() + visualizza_locker(): return Locker[] + visualizza_celle(idLocker): return Cella[] + cambia_stato_locker(id) + cambia_stato_cella(id) + cerca_locker(nome,categoria, codice): return Locker[] + gestisci_donazioni() + gestisci_segnalazioni() + analytics() + logout() + crea_locker() + affitto_cella_negozio()

Table 2: Dettaglio classe Operatore

L'operatore può visualizzare i locker e le celle associate, cambiare lo stato dei locker (attivo, inattivo, manutenzione), cercare locker specifici tramite nome, categoria o codice, gestire le donazioni ricevute, processare le segnalazioni inviate dagli utenti, visualizzare analytics sui dati di utilizzo del sistema, creare nuovi locker e affittare celle ai negozi per scopi commerciali.

Locker

La classe Locker rappresenta l'unità fisica principale del sistema UrbanLock. Ogni locker è caratterizzato da un identificativo univoco, un nome descrittivo, una categoria che ne definisce la tipologia d'uso e uno stato che indica la sua disponibilità operativa. I locker possono essere di diverse categorie (es. Deposito effetto personali, ciclismo, pet-friendly...) e possono trovarsi in diversi stati (online, offline, in manutenzione).

Locker

- IdLocker: int
- NomeLocker: string
- CategoriaLocker: enum
- StatoLocker: enum

- + aggiornaStato()
- + getCelle():return Cella[]

Table 3: Dettaglio classe Locker

Il locker può aggiornare il proprio stato in base alle condizioni operative e fornisce l'accesso alle celle che lo compongono attraverso il metodo getCelle(). Un locker contiene sempre almeno una cella e può essere visualizzato sia dagli utenti che dagli operatori.

Cella

La classe Cella rappresenta le singole unità di stoccaggio all'interno di un locker. Ogni cella è identificata univocamente (idCella) e possiede uno stato che indica se è libera, occupata, online, offline o in manutenzione

Cella

- IdCella: int
- StatoCella: enum

- + aggiornaStato()

Table 4: Dettaglio classe Cella

La cella può aggiornare dinamicamente il proprio stato in risposta alle operazioni dell'utente (apertura, chiusura, noleggio) o alle azioni dell'operatore (manutenzione, sblocco). Una cella appartiene sempre e solo ad un locker specifico.

Donazione

La classe Donazione gestisce le informazioni relative alle donazioni effettuate dagli utenti attraverso il sistema UrbanLock. Ogni donazione è identificata da un codice univoco (idDonazione), appartiene ad una specifica categoria, possiede una descrizione testuale, un tipo di donazione che può indicare diverse modalità (es. oggetti, vestiti, libri) e uno stato che traccia l'avanzamento della donazione nel processo di gestione (da visionare, accettata/rifiutata)

Donazione
- IdDonazione: int
- Categoria: enum
- descrizione: string
- ritiro: enum
- stato: enum
- data_creazione: Date
+ aggiornaStato()

Table 5: Dettaglio classe Donazione

La data di creazione (data_creazione: Date) viene automaticamente registrata al momento della creazione della donazione. Il metodo aggiornaStato() permette di modificare lo stato della donazione durante il suo ciclo di vita. Ogni donazione è creata da un solo utente ma può essere gestita da uno o più operatori nel corso del processo.

Segnalazione

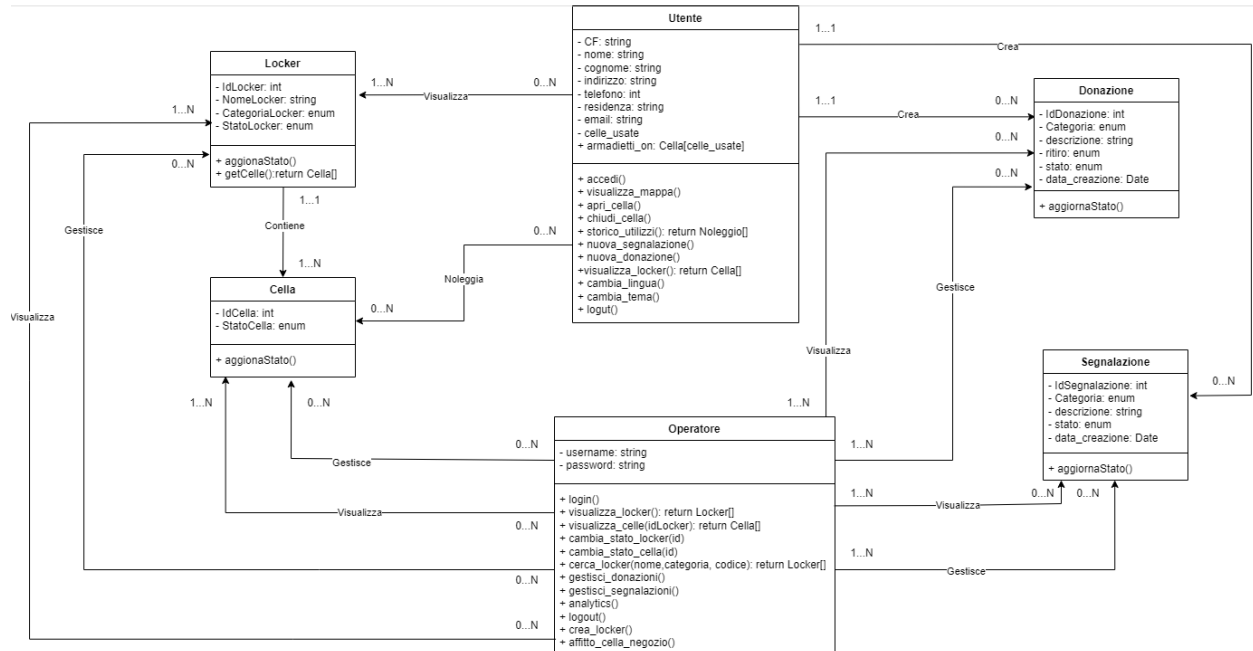
La classe Segnalazione permette agli utenti di comunicare problematiche, malfunzionamenti o richieste di assistenza relativamente all'utilizzo dei locker. Ogni segnalazione possiede un identificativo univoco (idSegnalazione), appartiene ad una categoria specifica che classifica il tipo di problema, include una descrizione dettagliata, ha uno stato che traccia l'avanzamento della gestione e registra la data di creazione.

Segnalazione
- IdSegnalazione: int - Categoria: enum - descrizione: string - stato: enum - data_creazione: Date
+ aggiornaStato()

Table 6: Dettaglio classe Segnalazione

Il metodo aggiornaStato() consente di modificare lo stato della segnalazione man mano che viene processata dagli operatori (es. da "nuova" a "in lavorazione" a "risolta"). Ogni segnalazione è creata da un singolo utente ma può essere visualizzata e gestita da più operatori fino alla sua risoluzione.

2.1 Diagramma delle classi complessivo



Per vedere una foto migliore, [clicca qui](#).

3. Dal class diagram alle API

Classe	Metodo	HTTP Method	URI + query params	Request	Response	Note
Utente	effettua_accesso(tipo, cf, ...)	POST	/api/v1/auth	{ "cf": "...", "tipo": "spid"	201{ "nome": "...", "cognome": "...", "email": "...", ... }	201{ "success": true, "user": {...}, "token": "... }
Utente	visualizza_mappa(filtri)	GET	/api/v1/lockers?type=...&category=...&distance=...	–	200{ "success": true, "lockers": [...] }	Endpoint usato per ottenere la lista locker (con o senza filtri) da mostrare su mappa/lista.
Utente	mostra_dettagli_locker(idLocker)	GET	/api/v1/lockers/ID	–	200{ "success": true, "locker": {..., "cells": [...]} }	Restituisce dettaglio locker e tipicamente anche le celle associate.
Utente	apertura_vano(lockerId, cellaId)	POST	/api/v1/deposits	{ "lockerId": "LOCK-001", "cellaId": "CELL-001" }	200/201{ "success": true, "deposit": {...} }	Richiede autenticazione ; in D2 è presente il caso “deposito non autenticato → 401”.
Utente	restituzione_vano(depositId, foto?)	PUT	/api/v1/deposits/ID	{ "action": "close", "photo": "..." }	200{ "success": true, "deposit": {..., "endDate": "..."} }	Chiusura del deposito/nolegg: in pratica aggiorna lo stato e registra metadati di restituzione (eventuale foto).

Utente	visualizza_storico_utilizzi()	GET	/api/v1/deposits?mine=true	—	200{ "success": true, "deposits": [...] }	Restituisce lo storico depositi/noleggi dell'utente autenticato.
Utente	crea_donazione(dati)	POST	/api/v1/donations	{ "categoria": "...", "descrizione": "...", "tipo": "...", "foto": "...", "foto": "..." }	201{ "success": true, "donation": { ... } }	Inserimento di una nuova donazione lato app, con allegati/descrizione.
Utente	visualizza_donazioni()	GET	/api/v1/donations?mine=true	—	200{ "success": true, "donations": [...] }	Cronologia donazioni dell'utente e stati di avanzamento.
Utente	crea_segna-lazione(dati)	POST	/api/v1/segnalazioni	{ "lockerId": "...", "categoria": "...", "descrizione": "...", "foto": "...", "foto": ["..."] }	201{ "success": true, "segnalazione": { ... } }	Crea una segnalazione associata a una postazione/locker.
Utente	visualizza_segnalazioni()	GET	/api/v1/segnalazioni?mine=true	—	200{ "success": true, "segnalazioni": [...] }	Elenco segnalazioni dell'utente (stato e dettagli principali).
Utente	visualizza_notifiche()	GET	/api/v1/notifications?mine=true	—	200{ "success": true, "notifications": [...] }	Restituisce notifiche ricevute dall'utente (donazioni/segnalazioni/eventi).
Utente	log_out()	—	—	—	—	Logout tipicamente gestito lato frontend eliminando il token.

Operatore	effettua_accesso(username, password)	POST	/api/v1/auth	{ "username": "...", "password": "..." }	200{ "success": true, "token": "..." }	Variante operatore: in pratica stesso "dominio auth", credenziali diverse; token abilita funzioni admin.
Operatore	lista_lockers(filtri)	GET	/api/v1/lockers?status=...&type=...	–	200{ "success": true, "lockers":[...] }	Usata in console per elenco filtrabile e gestione stati.
Operatore	crea_locker(datiLocker)	POST	/api/v1/lockers	{ "name": "...", "type": "...", "category": "...", "posizione": {...} }	201{ "success": true, "locker": {...} }	Creazione nuova postazione/locker dal pannello admin.
Operatore	cambia_stato_locker(id, stato)	PUT	/api/v1/lockers/ID	{ "status": "online" }	offline	manutenzione, "reason": "..." }
Operatore	lista_donazioni(filtri)	GET	/api/v1/donations?status=...&dateFrom=...	–	200{ "success": true, "donations":[...] }	Vista operatore: elenco richieste con filtri (stato/data/postazione/categoria).
Operatore	aggiorna_stato_donazione(id, stato)	PUT	/api/v1/donations/ID	{ "status": "...", "note": "..." }	200{ "success": true, "donation": {...} }	Cambio stato e gestione pratica dal dettaglio console.
Operatore	lista segnalazioni(filtri)	GET	/api/v1/segnalazioni?status=...&priority=...	–	200{ "success": true, "segnalazioni":[...] }	Vista operatore: coda segnalazioni con filtri.
Operatore	aggiorna segnalazione(id, stato/priorità/assegnazione)	PUT	/api/v1/segnalazioni/ID	{ "status": "...", "priority": "...", "assignedTo": ... }	200{ "success": true, "segnalazione": {...} }	Endpoint "unico" per aggiornare i campi di

	)			"..." }	e": {...} }	moderazione.
Operatore	genera_report(filtri)	GET	/api/v1/reports?type=...&dateFrom=...&dateTo=...	–	200{ "success": true, "report": {...} }	In D2 si parla di analytics e aggregazioni; l'implementazione concreta può esporre una risorsa "reports".
Operatore	esporta_report(formato, filtri)	GET	/api/v1/reports/export?format=pdf	xlsx&...'`	–	200file